Mind Stone

Technical Sheet — Participant Guide

Build a RAG Chatbot for DZShop

Data Space Club — ENSSEA — Algeria

November 6, 2026

Quick Summary

Problem: Build a Retrieval-Augmented Generation (RAG) chatbot that answers customer questions about the company DZShop accurately and faithfully.
Scoring: Application Quality (50%) + Presentation (20%) + Answers to Evaluation Questions (30%).
Submission: A public GitHub repository containing your code, notebooks, RAG architecture, and answers to the evaluation questions.

Contents

1 The Problem **1**

2 The Dataset — Knowledge Base **1**
2.1 Structure of the Knowledge Base 1
2.2 What Each File Contains 2

3 Files Provided **2**

4 Scoring Distribution **3**

5 Submission Format **3**

6 Conditions and Rules **3**
6.1 Technology Requirements 3
6.2 GitHub Repository 4
6.3 Notebooks and Code Files 4

7 Suggested Workflow **5**

8 Tips for Success **5**

9 Submission Checklist **5**

10 Evaluation Summary **6**

1 The Problem

You are tasked with building a Retrieval-Augmented Generation (RAG) chatbot for DZShop, an Algerian e-commerce company. The chatbot will serve as a customer-facing assistant that answers questions about the company's products, services, payment methods, shipping, returns, and more.

Why this matters: Real customers of DZShop are normal people (not technical). When they want to ask about payment options, delivery times, refund processes, or product availability, they need a quick, accurate, and truthful answer. A RAG chatbot retrieves relevant information from the company's documentation and generates a response — without hallucinating.

Your goal: Build a RAG pipeline that:

- Reads the company's knowledge base (provided as Markdown files).
- Retrieves the most relevant passages for a given question.
- Generates a clear, accurate, and faithful answer.

The chatbot must not invent information. Every answer must be grounded in the provided documents. Hallucinations will be heavily penalized.

2 The Dataset — Knowledge Base

You are given a collection of Markdown (.md) files that constitute the complete documentation of DZShop. These files describe every aspect of the company: who they are, what they sell, how customers can pay, how orders are shipped, how returns work, and more.

2.1 Structure of the Knowledge Base

The knowledge base is organized into 9 folders containing 32 Markdown files:

Folder	# Files	Content
about/	3	Company overview, store locations, contact info
products/	4	Categories, payment methods, shipping options, returns
orders/	4	How to order, tracking, modifying, delivery times
account/	3	Account creation, login help, password reset
payments/	4	Payment methods, refunds, installments, failures
shipping/	4	Zones, fees, same-day delivery, international
returns/	3	Return policy, refund timeline, exchange policy
faq/	4	General, shipping, payment, account FAQs
promotions/	3	Current offers, coupon codes, loyalty program

2.2 What Each File Contains

Each Markdown file is a self-contained document covering a specific topic. For example:

- `about/company_overview.md` — Company history, mission, leadership, capital.
- `products/categories.md` — Full product catalog with prices.
- `payments/installment_plans.md` — 3x, 6x, 12x installment details.
- `faq/general_faq.md` — 20 common questions and answers.

Important

The knowledge base contains specific numbers, dates, and policies. Your chatbot must extract these exactly — not approximately. For example, the return window is exactly 7 days, not "about a week".

3 Files Provided

File/Folder	Description
knowledge_base/	The 32 Markdown files (9 folders) describing DZShop
public_questions.csv	10–15 public questions for local testing
technical_sheet.pdf	This guide

4 Scoring Distribution

Your final grade is composed of three parts:

Component	Weight	What's Evaluated
Application Quality (Code, Repo, Architecture)	50%	Code quality, RAG pipeline design, GitHub repository
Presentation (Live Demo)	20%	Live demo of your chatbot (final day)
Answers to Evaluation Questions	30%	Accuracy and faithfulness of answers to the organizers' evaluation questions

5 Submission Format

Instead of a CSV leaderboard file, your submission consists of the following required deliverables:

- **GitHub Repository:** A public repository containing all source code, a `README.md`, a `requirements.txt` file, and a clear diagram/description of the RAG Architecture.
- **Notebooks and Code Files:** Jupyter notebooks (`.ipynb`) and Python scripts (`.py`) showing the full development process.
- **Architecture Design:** A clear explanation/diagram of the RAG pipeline (chunking strategy, embedding model, vector DB, LLM used).
- **Answers to Evaluation Questions:** A text file (e.g., `answers.txt` or `answers.json`) containing the chatbot's generated answers for the specific questions provided by the organizers (similar to `public_questions.csv`).

Rules:

- Answers must be grounded in the knowledge base — no hallucinations.
- Keep answers concise (1–3 sentences typically).
- If the answer is not in the knowledge base, answer: "I don't have this information in the available documentation."
- Answers must be plain text (no HTML, no Markdown).

6 Conditions and Rules

6.1 Technology Requirements

Allowed Technologies

You must use free and open-source technologies only. This includes:

- **Embedding models:** Any open-source model (e.g., all-MiniLM-L6-v2, sentence-transformers).
- **Vector databases:** ChromaDB, FAISS, Qdrant, Weaviate, Milvus (open-source versions).
- **LLMs:** Any open-source model (e.g., Llama 3, Mistral, Qwen via Ollama, vLLM, or HuggingFace).
- **Frameworks:** LangChain, LlamaIndex, Haystack, or custom code.
- **Languages:** Python (recommended), JavaScript/TypeScript.

NOT Allowed

- Paid APIs (OpenAI GPT-4, Anthropic Claude, Google Gemini).
- Closed-source proprietary models.
- Cloud services that require payment (e.g., Pinecone paid tier).
- Pre-built RAG APIs (e.g., OpenAI Assistants API).

6.2 GitHub Repository

You must provide a public GitHub repository containing:

- All source code (Python scripts, notebooks).
- A `README.md` explaining your approach.
- Instructions to run your chatbot locally.
- A `requirements.txt` or `environment.yml` file.
- Your RAG pipeline architecture (diagram or description).

6.3 Notebooks and Code Files

You must submit:

- Jupyter notebooks (`.ipynb`) showing your development process.
- Python scripts (`.py`) for the final pipeline.
- Any configuration files (e.g., prompt templates, chunking strategies).

Repository Naming
Name your repository: mind-stone-team-X (replace X with your team number).

7 Suggested Workflow

Phase	Task	Time
1	Explore the knowledge base (32 files)	1 hour
2	Choose embedding model + vector DB	30 min
3	Chunk the documents (experiment with size)	1 hour
4	Build the retrieval pipeline	1.5 hours
5	Choose LLM + write prompt template	1 hour
6	Test on public questions	1 hour
7	Optimize (chunking, retrieval, prompting)	2 hours
8	Generate answers for the provided evaluation questions	30 min
9	Prepare GitHub repo + documentation	1 hour
	Total	9.5 hours

8 Tips for Success

- **Chunking matters:** Experiment with different chunk sizes (256, 512, 1024 tokens).
- **Retrieval quality:** Use semantic search + keyword search (hybrid) for better results.
- **Prompt engineering:** Tell the LLM explicitly: "Answer only from the retrieved context. If not found, say so."
- **Multi-hop questions:** Some questions require combining info from 2 files. Plan for it.
- **Trick questions:** Some questions have no answer in the KB. Detect and respond accordingly.
- **Test locally:** Use the public questions to validate before submitting.

9 Submission Checklist

Before submitting, verify:

- ✔ GitHub repository is public and contains all required files.
- ✔ `README.md` explains your approach.
- ✔ `requirements.txt` is included.
- ✔ Notebooks (`.ipynb`) and scripts (`.py`) are clean and reproducible.
- ✔ Architecture diagram/description of the RAG pipeline is included.
- ✔ Answers to evaluation questions are included (`answers.txt` or `answers.json`).
- ✔ Answers are concise and grounded in the knowledge base.

10 Evaluation Summary

Component	Weight
Application Quality (Code, Repo, Architecture)	50%
Presentation (Live Demo)	20%
Answers to Evaluation Questions	30%

Good luck!

Build a chatbot that DZShop customers can trust.